Practical No. 2: Implement assignment based on feature extraction techniques for effective machine learning model building

Name: Shantanu Yadav

Roll No: 75

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
df = pd.read_csv('Cancer_Data.csv')
df.head()
```

Out[2]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
0	842302	M	17.99	10.38	122.80	1001.0
1	842517	M	20.57	17.77	132.90	1326.0
2	84300903	M	19.69	21.25	130.00	1203.0
3	84348301	M	11.42	20.38	77.58	386.1
4	84358402	M	20.29	14.34	135.10	1297.0

5 rows × 33 columns

In [3]:
```
print('Shape of dataset:', df.shape)
print('\nDataset information:')
df.info()
```

```
Shape of dataset: (569, 33)

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   id                       569 non-null    int64  
 1   diagnosis                569 non-null    str    
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), str(1)
memory usage: 147.4 KB
```

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
Unnamed: 32                569
dtype: int64
```

In [5]:
```
df = df.drop(columns=['id'], errors='ignore')
df = df.dropna(axis=1, how='all')

df['diagnosis'] = df['diagnosis'].map({'B': 0, 'M': 1})

df.head()
```

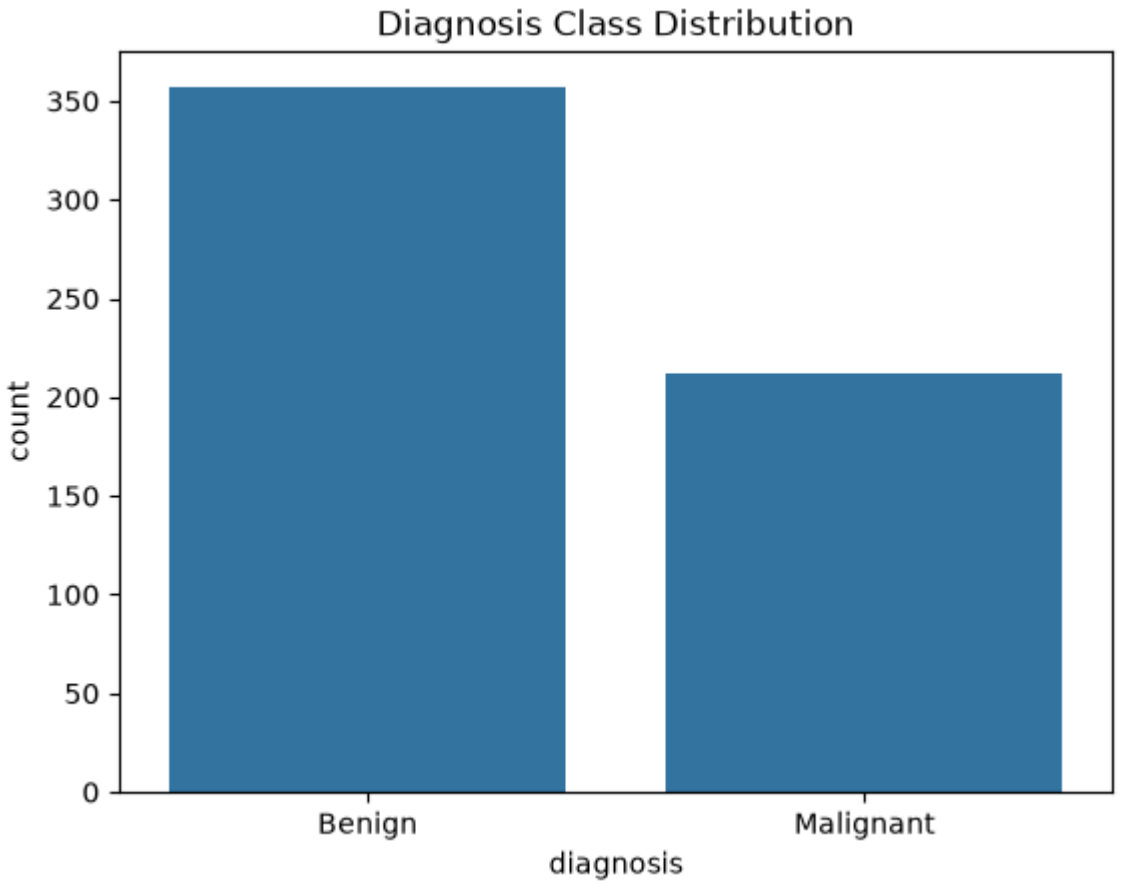
Out[5]:

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothnes
0	1	17.99	10.38	122.80	1001.0	
1	1	20.57	17.77	132.90	1326.0	
2	1	19.69	21.25	130.00	1203.0	
3	1	11.42	20.38	77.58	386.1	
4	1	20.29	14.34	135.10	1297.0	

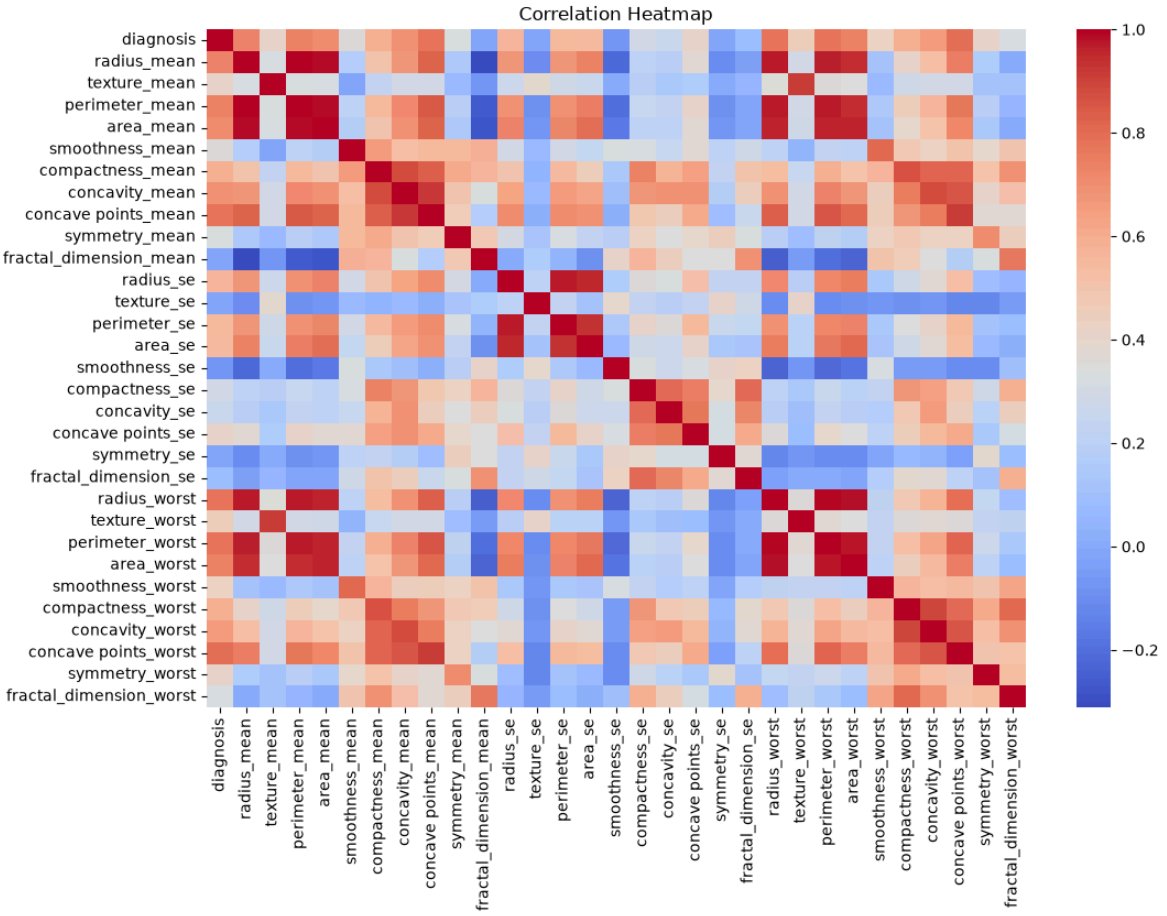
5 rows × 31 columns

In [6]:
```
sns.countplot(x='diagnosis', data=df)
plt.xticks([0, 1], ['Benign', 'Malignant'])
```

```
plt.title('Diagnosis Class Distribution')
plt.show()
```



In [7]:
```
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



In [8]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print('Training data shape:', X_train_scaled.shape)
print('Testing data shape:', X_test_scaled.shape)
```

```
Training data shape: (455, 30)
Testing data shape: (114, 30)
```

Feature Extraction using PCA

In [9]:
```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

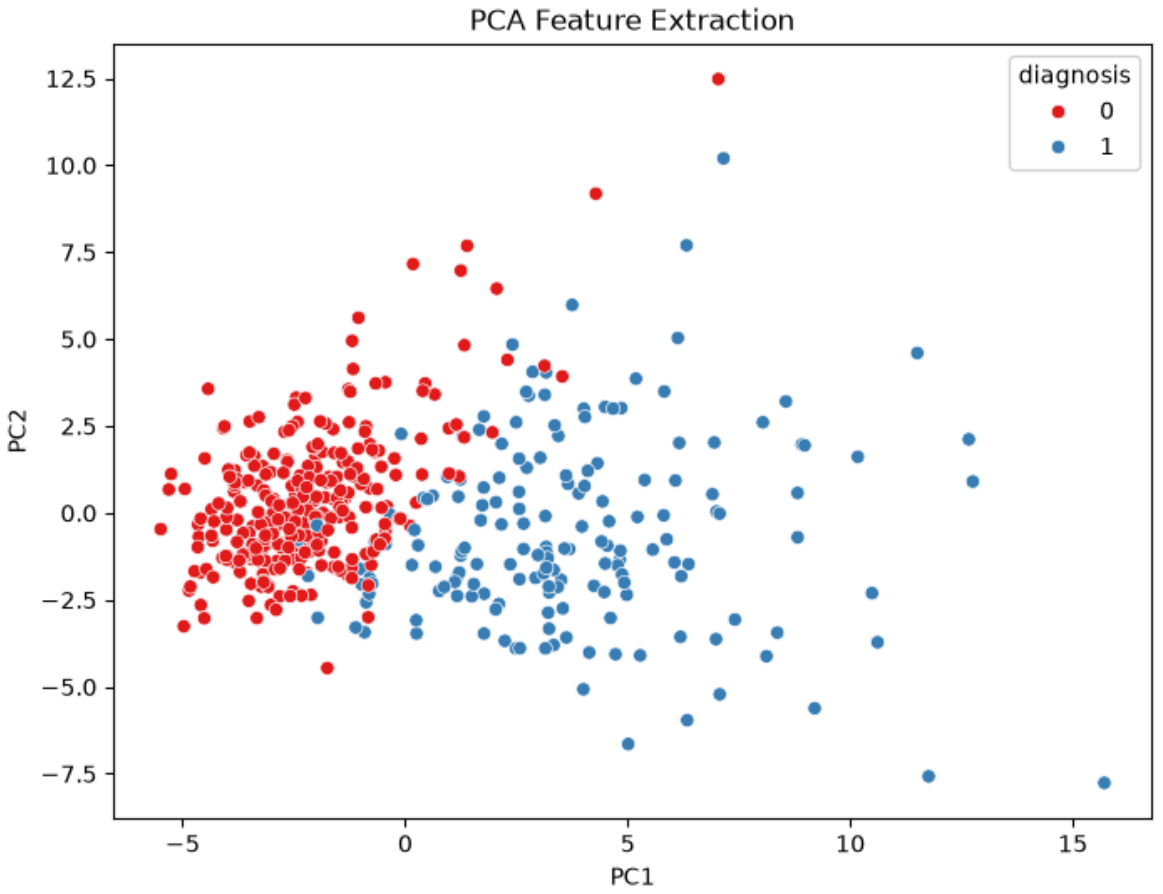
print('Original number of features:', X_train_scaled.shape[1])
print('Extracted number of features:', X_train_pca.shape[1])
```

```
print('Explained variance ratio:', pca.explained_variance_ratio_)
print('Total variance explained:', round(pca.explained_variance_ratio_.su
```

```
Original number of features: 30
Extracted number of features: 2
Explained variance ratio: [0.44593522 0.18545255]
Total variance explained: 0.6314
```

In [10]:
```
pca_df = pd.DataFrame(X_train_pca, columns=['PC1', 'PC2'])
pca_df['diagnosis'] = y_train.values

plt.figure(figsize=(8, 6))
sns.scatterplot(x='PC1', y='PC2', hue='diagnosis', data=pca_df, palette='
plt.title('PCA Feature Extraction')
plt.show()
```



Model Building using Extracted Features

In [11]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(random_state=42)
model.fit(X_train_pca, y_train)

y_pred = model.predict(X_test_pca)
model
```

Out[11]:

▾ LogisticRegression

▸ Parameters

▸ Fitted attributes

In [12]:
```
from sklearn.metrics import accuracy_score, confusion_matrix, classificat

accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print('Accuracy:', accuracy)
print('\nConfusion Matrix:')
print(conf_matrix)
print('\nClassification Report:')
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9473684210526315

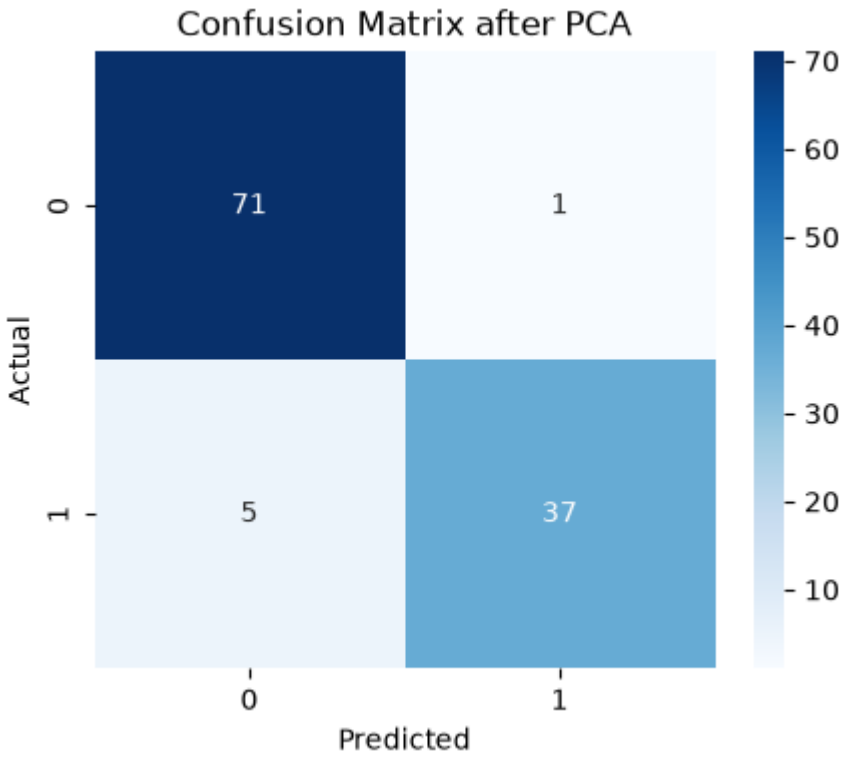
Confusion Matrix:
[[71  1]
 [ 5 37]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.99      0.96        72
           1       0.97      0.88      0.93        42

    accuracy                           0.95       114
   macro avg       0.95      0.93      0.94       114
weighted avg       0.95      0.95      0.95       114
```

In [13]:
```
plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix after PCA')
plt.show()
```



Feature Selection using SelectKBest

In [14]:
```
from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(score_func=f_classif, k=10)
X_train_best = selector.fit_transform(X_train_scaled, y_train)
X_test_best = selector.transform(X_test_scaled)

selected_features = X.columns[selector.get_support()]
print('Selected Features:')
print(selected_features)
```

```
Selected Features:
Index(['radius_mean', 'perimeter_mean', 'area_mean', 'concavity_mean',
       'concave points_mean', 'radius_worst', 'perimeter_worst', 'area_wor
st',
       'concavity_worst', 'concave points_worst'],
      dtype='str')
```

In [15]:
```
selected_model = LogisticRegression(random_state=42)
selected_model.fit(X_train_best, y_train)

selected_pred = selected_model.predict(X_test_best)
selected_accuracy = accuracy_score(y_test, selected_pred)

print('Accuracy using PCA features:', accuracy)
print('Accuracy using SelectKBest features:', selected_accuracy)
```

```
Accuracy using PCA features: 0.9473684210526315
Accuracy using SelectKBest features: 0.956140350877193
```